IAM-07: Audit Logging and Compliance

> **Series:** IAM — IAM Administration | **Notebook:** 7 of 12 | **Created:** January 2026 | **Last Updated:** 10/19/2026

Meeting Regulatory and Security Requirements

Audit logging is essential for compliance (SOC2, SOX, HIPAA, PCI-DSS) and security operations. This notebook covers querying audit logs, building compliance reports, and conducting access reviews.

Table of Contents

1. Audit Log Fundamentals
2. Authentication Event Monitoring
3. Authorization and Access Events
4. Configuration Change Tracking
5. Compliance Reporting Patterns
6. Access Review Workflows
7. Building Compliance Dashboards

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail — Grail-based audit logging is active by default in every environment
Permissions	`ALLOW storage:system:read WHERE storage:event.kind = "AUDIT_EVENT";` and `ALLOW storage:buckets:read WHERE storage:bucket-name = "dt_system_events";` — plus `ALLOW iam:users:read;` to resolve the `user.id` UUIDs in audit records to user details through the IAM API
Account audit log	The `account-company-info` account permission, to review account-level IAM changes in Account Management
Data Retention	Environment audit events are kept for one year; plan exports if a framework needs longer

1. Audit Log Fundamentals

Dynatrace audit logs capture all security-relevant events in your environment.

What Gets Logged

Category	Events
Authentication	Sign-ins (`event.type == "LOGIN"`), including SSO sign-ins — MFA itself happens in your IdP
Authorization	Access attempts, permission checks
User Management	User creation, modification, deletion
Group Management	Group changes, membership updates
Policy Changes	Policy creation, modification, deletion
Token Operations	Token creation, revocation, usage
Configuration	Settings changes, API calls

Audit Log Fields

Field	Description	Example
`timestamp`	Event time	`2026-01-26T14:30:00Z`
`event.kind`	Always `AUDIT_EVENT` for audit records	`AUDIT_EVENT`
`event.type`	Action	`LOGIN`, `GET`, `POST`, `CREATE`, `DELETE`
`event.outcome`	Result — `success` / `failure`, or an HTTP status	`success`, `403`
`event.provider`	Component that recorded the event	`API_GATEWAY`, `SETTINGS`, `API_TOKEN`
`user.id`	Acting user (a UUID, not an email address)	`37e52d7a-…`
`user.organization`	Organization of the acting user	`DYNATRACE`
`authentication.type` / `authentication.grant.type`	How the caller authenticated	`OAUTH2`, `TOKEN`
`resource`	The API path or object acted on	`/platform/document/v1/…`
`origin.address`	Client IP address	`203.0.113.10`
`request.source`	Where the request came from	`BROWSER`, `DT_SERVERLESS`, `OTHER`

> **Two audit records, two places.** Activity *inside this environment* — sign-ins, API calls, settings and token changes — is recorded as `AUDIT_EVENT` records in `dt.system.events`, which every query in this notebook reads. **Account-level IAM changes** — groups, memberships, policies, boundaries — are recorded in the account audit log (Account Management > Settings > Audit log), not here.

> **Not every action is recorded yet.** Dynatrace states that audit events from inside platform services are not recorded yet — its examples are an app installed by AppEngine and a document being shared. An evidence report that needs those actions cannot be built from `dt.system.events` today; say so in the report rather than presenting an empty result as "no activity".
>
> **Sources:** Audit logs on Grail (DT docs) — *"Grail-based audit logging is active by default in all Dynatrace environments."*; *"Audit events from within platform services, such as an app being installed by AppEngine or a document being shared, aren't recorded yet."*; *"Audit events are stored as dt.system.events in the dt_system_events bucket, retained for one year"*; the page lists `ALLOW iam:users:read;` for looking up user details from the user ID. Account Management audit logs (DT docs) — *"Dynatrace provides audit logs of changes to your account-level identity and access (IAM) management settings"*.

Retention Requirements

Environment audit events are kept for one year. Whether that is enough depends on the framework and on how your auditors read it. In community practice, teams commonly plan around figures like the ones below — confirm the period that applies to you with your compliance or legal team before sizing an export.

Compliance	Commonly planned retention (community practice)
SOC2	1 year
SOX	7 years
HIPAA	6 years
PCI-DSS	1 year, with recent months immediately available
GDPR	As long as necessary

2. Authentication Event Monitoring

Track who is accessing your Dynatrace environment and how.

```
// Authenticated API calls by credential type (24 h)
// Retitled 10/12/2026: authentication.type is set on every audited request,
so this is API traffic
// (about 31,700 calls a day on the validation tenant), not sign-ins. Sign-ins
are event.type == "LOGIN".
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "AUDIT_EVENT"
| filter isNotNull(authentication.type)
| summarize events = count(), by:{authentication.type, event.outcome}
| sort events desc
```

```
// Failed sign-in attempts - critical for security monitoring
// Corrected 09/24/2026: a failed sign-in is event.type == "LOGIN" with
event.outcome == "failure".
// The previous "any non-2xx outcome" predicate counted 404 GETs and even 304
Not Modified.
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter event.type == "LOGIN" and event.outcome == "failure"
| summarize failures = count(), by:{user.id, origin.address}
| sort failures desc
| limit 25
```

```
// Sign-in activity by hour (successful and failed interactive logins)
// Corrected 10/12/2026: this cell counted isNotNull(authentication.type),
which is every authenticated API
// call: 31,712 events in 24 h on the validation tenant, of which 4 were sign-
ins.
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter event.type == "LOGIN"
| makeTimeseries logins = count(), by:{event.outcome}, interval:1h
```

Authentication Patterns to Monitor

Pattern	Indicates	Query Focus
Multiple failed logins	Brute force attempt	Failed + same user
Off-hours logins	Unusual access	Timestamp analysis
New location logins	Account compromise	IP/location change
Rapid successive logins	Automation or sharing	Time between logins
Login after offboarding	Process failure	User status check

```
// Off-hours activity detection (outside 08:00-18:00)
// formatTimestamp renders the hour in the query's timezone (the notebook or
user timezone setting), not
// necessarily UTC: set the timezone to the one your business hours are
defined in before reading the result.
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| fieldsAdd hour = formatTimestamp(timestamp, format: "HH")
| filter toLong(hour) &lt; 8 or toLong(hour) &gt;= 18
| summarize off_hours_events = count(), by:{user.id}
| sort off_hours_events desc
| limit 25
```

3. Authorization and Access Events

Track permission checks and access attempts to sensitive resources.

> **Group, policy, boundary and binding changes are account-level, and this environment's audit events are not their change log.** *"Dynatrace provides audit logs of changes to your account-level identity and access (IAM) management settings" — "Administrators can view these logs in Account Management > Settings > Audit log"*, where they are kept for up to ten years (also queryable through the

> Account Audits API). The same page says *"The Account Management audit log doesn't yet cover every account-level change. Some IAM configuration and login events aren't recorded."* The query below shows IAM API calls made **through this environment** — useful for spotting automation, not a change log. On the validation tenant, 30 days of these events held no group or policy change at all: only user lookups and scheduled `/lookups/iam_*` file uploads, which the query now excludes.
>
> **Sources:** Account Management audit logs (DT docs).

```
// Access denied events - indicates permission issues or attacks
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter event.outcome == "403"
| summarize denials = count(), by:{user.id, resource}
| sort denials desc
| limit 25
```

```
// IAM API activity in this environment (account-level IAM changes: see the
note above)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "iam")
| filter not startsWith(resource, "/lookups/")
| summarize changes = count(), by:{user.id, event.type}
| sort changes desc
```

Access Event Categories

Event Type	Security Relevance	Monitoring Priority
Access denied	Permission misconfiguration or probe	High
Privilege escalation	Critical security event	Critical
Cross-environment access	Boundary violation	High
Admin action	Privileged operation	Medium
Bulk data access	Exfiltration risk	High

4. Configuration Change Tracking

Track changes recorded in this environment. Account-level IAM changes (groups, policies, boundaries, bindings) are in the account audit log — see the note in Section 3.

User accounts are not in these records. Users are created, invited and removed at account level, so an environment `AUDIT_EVENT` never describes a user lifecycle change; the `CREATE` / `DELETE` events below are settings objects, tokens and lookup files. For joiner-and-leaver evidence, use the account audit log, which covers *"Group memberships"*, together with your IdP's provisioning log when users arrive by SCIM (IAM-06).

> **Sources:** Account Management audit logs (DT docs) — *"Dynatrace provides audit logs of changes to your account-level identity and access (IAM) management settings."*

```
// Create / delete activity in this environment — settings objects, API and
ActiveGate tokens,
// lookup files. NOT user accounts: users are account-level, so no AUDIT_EVENT
here records a user
// being created or deleted (retitled 10/14/2026; on the validation tenant 30
d of CREATE/DELETE was
// 11,080 SETTINGS, 390 lookup-file, 102 token, 25 API_GATEWAY, 3 CLASSIC_API
and 1 SLO events).
// event.provider says which kind of object each row is.
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"CREATE", "DELETE"})
| fields timestamp, user.id, event.provider, event.type, resource,
event.outcome
| sort timestamp desc
| limit 50
```

```
// IAM API activity in this environment — recent calls (group membership
itself is account-level)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "iam")
| filter not startsWith(resource, "/lookups/")
| fields timestamp, user.id, event.type, resource
| sort timestamp desc
| limit 50
```

```
// IAM API activity in this environment, by user (policy changes themselves:
account audit log)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "iam")
| filter not startsWith(resource, "/lookups/")
| summarize iam_api_calls = count(), by:{user.id}
| sort iam_api_calls desc
```

```
// Token lifecycle: API and ActiveGate token creation, update and deletion
// Corrected 09/24/2026: token lifecycle events carry event.provider API_TOKEN
/ ACTIVE_GATE_TOKEN,
// and their resource is the token ID, so the previous contains(resource,
"token") filter missed
// every one of them and matched only scheduled /lookups/iam_tokens uploads.
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.provider, {"API_TOKEN", "ACTIVE_GATE_TOKEN"})
| fields timestamp, user.id, event.provider, event.type, resource,
event.outcome
| sort timestamp desc
| limit 50
```

Change Tracking Summary

Change Type	Audit Requirement	Review Frequency
User lifecycle	Log all	Weekly
Group changes	Log all	Weekly
Policy changes	Log + approve	On change
Admin actions	Log + review	Daily

Change Type	Audit Requirement	Review Frequency
Token operations	Log all	Monthly

5. Compliance Reporting Patterns

Standard queries for compliance frameworks.

SOC2 - Access Control Evidence

SOC2 requires evidence of access controls and monitoring.

```
// SOC2: Users who signed in during the period
// Corrected 09/24/2026: this cell queried `fetch logs | filter
matchesPhrase(log.source, "audit")`,
// which matches nothing on most tenants, or unrelated file-based audit logs
(for example database
// .aud files). Platform audit records are AUDIT_EVENT records in
dt.system.events.
fetch dt.system.events, from:-90d
| filter event.kind == "AUDIT_EVENT"
| filter event.type == "LOGIN" and event.outcome == "success"
| summarize {last_login = takeMax(timestamp), login_count = count()}, by:
{user.id}
| sort last_login desc
| limit 500
```

```
// SOC2: Administrative action log
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-90d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"})
// Corrected 10/12/2026: many POSTs change nothing. DQL queries, permission
checks, verify/autocomplete/preview
// calls and classic token lookups are reads; app-state saves are UI
bookkeeping; workflow runs (…/run) are
// executions, not edits; /lookups/ uploads are a scheduled job. Over 30 days
on the validation
// tenant these were 177,504 of 198,069 write-verb events (90 %). Check the
remaining resource
// list on your tenant for further read-style POSTs.
| filter not(contains(resource, "/query:") or contains(resource, "effective-
permissions")
             or endsWith(resource, "/lookup") or contains(resource, "/verify")
             or contains(resource, "/autocomplete") or contains(resource,
"filter-preview")
             or endsWith(resource, "/run") or startsWith(resource,
"/platform/state/v1/user-app-states/")
             or contains(resource, "lookup:upload") or startsWith(resource,
"/lookups/"))
| fields timestamp, user.id, user.organization, event.type, resource,
event.outcome
| sort timestamp desc
| limit 100
```

SOX - Segregation of Duties

SOX requires segregation of duties and change authorization.

```
// SOX: Privileged user activity (write operations)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"})
// Corrected 10/12/2026: many POSTs change nothing. DQL queries, permission
checks, verify/autocomplete/preview
// calls and classic token lookups are reads; app-state saves are UI
bookkeeping; workflow runs (…/run) are
// executions, not edits; /lookups/ uploads are a scheduled job. Over 30 days
on the validation
// tenant these were 177,504 of 198,069 write-verb events (90 %). Check the
remaining resource
// list on your tenant for further read-style POSTs.
| filter not(contains(resource, "/query:") or contains(resource, "effective-
permissions")
             or endsWith(resource, "/lookup") or contains(resource, "/verify")
             or contains(resource, "/autocomplete") or contains(resource,
"filter-preview")
             or endsWith(resource, "/run") or startsWith(resource,
"/platform/state/v1/user-app-states/")
             or contains(resource, "lookup:upload") or startsWith(resource,
"/lookups/"))
| summarize privileged_actions = count(), by:{user.id, event.type}
| sort privileged_actions desc
| limit 25
```

```
// SOX: Configuration changes requiring approval
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "settings")
// Corrected 10/12/2026: effective-permissions:resolve is a permission check
sent as POST, not a change.
// It was 1,149 of 2,314 matching events over 30 days on the validation
tenant.
// The settings service also records changes with event.provider ==
"SETTINGS", where resource is the
// settings object rather than an API path, so this filter does not select
them.
| filter not contains(resource, "effective-permissions")
| fields timestamp, user.id, event.type, resource, event.outcome
| sort timestamp desc
| limit 50
```

HIPAA - PHI Access Logging

HIPAA requires logging of access to protected health information.

Most reads of data stored in Grail are DQL queries, so the report below counts query executions as well as `GET` calls. A query's audit record names the caller, the app and the API path. On the validation tenant it did not carry the query text or the records the query

returned. Use buckets and security context to limit what a query can read: the access model sets that limit, and the audit log only records that the query ran.

```
// HIPAA: data access events — API reads (GET) and Grail queries (POST
…/query:execute)
// Corrected 10/12/2026: running a DQL query is a POST to query:execute, so a
GET-only filter never showed
// who queried logs, spans or business events. That was 112,000 events over 30
days on the validation tenant.
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter event.type == "GET" or contains(resource, "/query:execute")
| summarize reads = count(), by:{user.id, resource}
| sort reads desc
| limit 50
```

```
// HIPAA: Access anomalies - bulk query detection against each caller's own baseline
// Corrected 10/13/2026: this cell flagged any user with more than 1,000 audit events in 7 days, which
// is every regular browser user (10 users on the validation tenant). A dashboard load alone runs many
// queries. This version counts Grail query executions per hour and flags callers whose busiest hour
// was at least 5x their typical (median) hour and above 100 queries; it returned 4 browser users on
// the validation tenant, peaks of 389-1,463 queries/hour against medians of 4-109. Tune both numbers.
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in `logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type (GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"), authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path), origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter contains(resource, "/query:execute")
| fieldsAdd hour = bin(timestamp, 1h)
| summarize queries = count(), by:{user.id, request.source, hour}
| summarize {peak_per_hour = max(queries), typical_per_hour = median(queries), active_hours = count()}, by:{user.id, request.source}
| fieldsAdd peak_vs_typical = round(toDouble(peak_per_hour) / typical_per_hour, decimals: 1)
| filter peak_per_hour &gt; 100 and peak_vs_typical &gt;= 5
| sort peak_vs_typical desc
| limit 25
```

PCI-DSS - Cardholder Data Access

PCI-DSS requires tracking access to cardholder data.

```
// PCI-DSS: Failed access attempts (potential attack indicator)
// Corrected 10/13/2026: access failures are 401 (not authenticated), 403 (not
authorized) and failed
// sign-ins. The previous "failure or any status &gt;= 400" predicate was
mostly 404 Not Found, 400 Bad
// Request and 5xx server errors: over 30 days on the validation tenant 3,034
of its events were 404s,
// against 43 401s, 411 403s and 3 failed sign-ins. Those are not access
attempts.
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.outcome, {"401", "403"}) or (event.type == "LOGIN" and
event.outcome == "failure")
| fieldsAdd failure_kind = if(event.outcome == "401", "unauthenticated (401)",
else: if(event.outcome == "403", "forbidden (403)", else: "failed sign-in"))
| summarize failures = count(), by:{user.id, origin.address, failure_kind}
| sort failures desc
| limit 50
```

Compliance Query Reference

Framework	Key Reports	Frequency
SOC2	User access, admin actions, logical access	Quarterly
SOX	Privileged users, change authorization	Quarterly
HIPAA	PHI access, anomaly detection	Monthly
PCI-DSS	Failed access, CDE access	Daily/Weekly
GDPR	Data access, consent tracking	On request

6. Access Review Workflows

Periodic access reviews are required by most compliance frameworks.

Access Review Process

Step	Action	Owner
1	Generate user access report	IAM Admin
2	Distribute to managers	IAM Admin
3	Review and certify access	Managers
4	Remove inappropriate access	IAM Admin
5	Document review completion	IAM Admin

```
// Access review: most and least active users in the window
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-180d
| filter event.kind == "AUDIT_EVENT"
| summarize {events = count(), last_seen = takeMax(timestamp)}, by:{user.id}
| sort last_seen asc
| limit 50
```

```
// Access review: High-activity users (potential over-privilege)
// Corrected 09/24/2026: this cell queried `fetch logs | filter
matchesPhrase(log.source, "audit")`,
// which matches nothing on most tenants, or unrelated file-based audit logs
(for example database
// .aud files). Platform audit records are AUDIT_EVENT records in
dt.system.events.
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| summarize action_count = count(), by:{user.id}
| sort action_count desc
| limit 50
```

Access Review Checklist

```
☐ All users verified as current employees
☐ Access levels appropriate for job function
☐ No separation of duties violations
☐ Privileged access justified and approved
☐ Inactive accounts identified for removal
☐ Service accounts still required
☐ Review documented and signed off
```

Review Frequency by Access Level

Access Level	Review Frequency	Approver
Viewer	Annually	Manager
Editor	Semi-annually	Manager + Security
Admin	Quarterly	Director + Security
Account Admin	Monthly	CISO

7. Building Compliance Dashboards

Create dashboards for ongoing compliance monitoring.

Dashboard Components

Section	Metrics	Chart Type
Authentication	Logins/failures over time	Line chart
Access Denied	Failed access attempts	Bar chart
User Changes	Create/delete/modify	Table
Admin Activity	Privileged actions	Table
Token Operations	Creates/revocations	Table

```
// Dashboard: Sign-in trend (for line chart)
// Corrected 09/24/2026: this cell queried `fetch logs | filter
matchesPhrase(log.source, "audit")`,
// which matches nothing on most tenants, or unrelated file-based audit logs
(for example database
// .aud files). Platform audit records are AUDIT_EVENT records in
dt.system.events.
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter event.type == "LOGIN" and event.outcome == "success"
| makeTimeseries logins = count(), interval:24h
```

```
// Dashboard: Failed access by day (for bar chart) — 401, 403 and failed sign-
ins
// Corrected 10/13/2026: "any status &gt;= 400" was mostly 404s and server
errors, not access failures.
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.outcome, {"401", "403"}) or (event.type == "LOGIN" and
event.outcome == "failure")
| fieldsAdd failure_kind = if(event.type == "LOGIN", "failed sign-in", else:
event.outcome)
| makeTimeseries failures = count(), by:{failure_kind}, interval:24h
```

```
// Dashboard: Recent administrative actions (for table)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"})
// Corrected 10/12/2026: many POSTs change nothing. DQL queries, permission
checks, verify/autocomplete/preview
// calls and classic token lookups are reads; app-state saves are UI
bookkeeping; workflow runs (…/run) are
// executions, not edits; /lookups/ uploads are a scheduled job. Over 30 days
on the validation
// tenant these were 177,504 of 198,069 write-verb events (90 %). Check the
remaining resource
// list on your tenant for further read-style POSTs.
| filter not(contains(resource, "/query:") or contains(resource, "effective-
permissions")
             or endsWith(resource, "/lookup") or contains(resource, "/verify")
             or contains(resource, "/autocomplete") or contains(resource,
"filter-preview")
             or endsWith(resource, "/run") or startsWith(resource,
"/platform/state/v1/user-app-states/")
             or contains(resource, "lookup:upload") or startsWith(resource,
"/lookups/"))
| fields timestamp, user.id, event.type, resource, event.outcome
| sort timestamp desc
| limit 25
```

Alert Thresholds

Configure alerts based on audit log patterns:

Condition	Threshold	Severity
Failed logins	> 5 in 10 min same user	Warning
Failed logins	> 20 in 10 min any	Critical
Admin action	Any outside business hours	Warning
Policy change	Any	Info
User deletion	Any	Info
Bulk data access	Busiest hour ≥ 5× the caller's typical hour (see the HIPAA bulk-query cell)	Warning

Next Steps

With compliance monitoring in place, proceed to multi-environment management:

Recommended Path

1. **IAM-08: Multi-Environment IAM** - Scale IAM across environments
2. **IAM-09: Troubleshooting Access Issues** - Debug permission problems

Compliance Checklist

Before moving on, ensure you have:

- [ ] Identified applicable compliance frameworks
- [ ] Created queries for required audit reports
- [ ] Established access review schedule
- [ ] Built compliance monitoring dashboard
- [ ] Configured alerts for critical events
- [ ] Documented audit log retention policy

Summary

In this notebook, you learned:

- Audit log fundamentals and retention requirements
- Authentication event monitoring patterns
- Authorization and access event tracking
- Configuration change monitoring
- Compliance-specific query patterns (SOC2, SOX, HIPAA, PCI-DSS)
- Access review workflows
- Compliance dashboard components

References

- Audit logs on Grail (DT docs)
- Account Management audit logs (DT docs)
- DQL Reference
- SOC2 Trust Principles

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.